

UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación paralela y distribuida

Facultad de Ingeniería

Sección 21

Santiago Solorzano Padilla



Parcial 1 - Consultoría “Hilo Conductor”

Joel Jaquez - 23369

Carlos Alburez - 23311

Fernando Hernández - 23645

GUATEMALA, 10 de septiembre 2026

Enlace al repositorio: <https://github.com/FerAHMz/examen-parcial-1.git>

1. Contexto y Datos

a. Los problemas que elegimos:

- i. De los 3 problemas que traía el equipo, escogimos 2, que fueron el del histograma masivo y la búsqueda de la ruta mínima en una red social. Los elegimos porque se dejan repartir de formas muy distintas. En el histograma, los datos se pueden repartir en pedazos que no dependen entre sí y cada trabajador puede contar su parte sin preguntarle nada a los demás. En la búsqueda de ruta mínima pasa lo contrario, porque aquí el recorrido avanza por niveles y no se puede empezar el siguiente nivel hasta terminar el anterior. Estos 2 problemas nos dejaron comparar qué tanto rinde paralelizar cuando el trabajo se separa fácil y qué tanto rinde cuando hay dependencias de por medio.

b. Problema 1 - Histograma masivo:

- i. **¿Qué hace el programa?:** El programa recibe una cantidad de temperaturas medidas entre -100 y 100 grados, las ordena de menor a mayor y después cuenta cuántas caen dentro de cada una de 100 cubetas. El ancho de cada cubeta sale de dividir el rango completo de temperaturas entre esas 100 cubetas.
- ii. **Propuesta secuencial original:** La solución secuencial trabaja en 2 etapas. La primera etapa ordena el arreglo con Merge Sort. Es el ordenamiento clásico de dividir y vencer que parte el arreglo a la mitad una y otra vez hasta llegar a pedazos de un solo elemento y después va uniendo esas mitades ya ordenadas. La segunda etapa construye el histograma, aquí el detalle a resaltar de la solución es que aprovecha que el arreglo ya viene ordenado. En lugar de revisar cada temperatura y buscar a qué cubeta pertenece la función avanza en un índice sobre el arreglo y otro sobre las cubetas al mismo tiempo. Mientras la temperatura actual siga cayendo dentro del rango de la cubeta activa la cuenta y avanza al siguiente dato. Cuando ya no cabe, pasa a la siguiente y recalcula los límites. Con esto el arreglo se recorre completo una sola vez.
- iii. **Datos de prueba:** Usamos un millón de temperaturas, esto porque es el tope que aguanta el arreglo del programa, fue declarado de forma estática con MAX igual a 1000000. Los valores se generan al azar entre -100 y 100 con dos decimales, es decir que hay 20001 valores distintos posibles y se usa una semilla fija tanto para secuencial como paralelo para garantizar que ambos tienen los mismos números en las pruebas.

- iv. **¿Cómo se guardan los datos en memoria?:** El programa usa 3 arreglos globales de 1 millón de números flotantes cada uno. El arreglo A guarda las temperaturas originales sin ordenar, temp guarda la copia que sí se ordena y tempAux es el arreglo auxiliar que Merge necesita para ir intercalando las mitades sin perder datos. El histograma en sí es un arreglo de 100 enteros.

c. **Problema 2 - Ruta mínima en una red social:**

- i. **¿Qué hace el programa?:** El programa busca la menor cantidad de conexiones que separan a 2 personas dentro de una red social. Por ejemplo si Ana quiere llegar a Hugo y no son amigos directos, el programa encuentra la cadena más corta de amigos que los une.
- ii. **Propuesta secuencial original:** La red se guarda como un grafo donde cada usuario tiene una lista con los identificadores de sus amigos. El recorrido se hace con Búsqueda en Amplitud que se conoce como BFS, este funciona como una cola, entonces comienza metiendo al usuario de origen y marcándolo como visitado. Después se saca al que está al frente de la cola, se revisan todos sus amigos y los que todavía no habían sido descubiertos se marcan y se meten al final de la cola. Eso se repite hasta encontrar el destino o hasta que la cola se vacíe. Además de la cola el algoritmo guarda 3 arreglos. El arreglo visitado dice si un usuario ya fue descubierto, para no procesarlo 2 veces. El arreglo invitadoPor guarda desde cuál usuario se llegó hasta cada quién y sirve para reconstruir el camino hacia atrás desde el destino. El arreglo distancia lleva la cuenta de cuántos saltos se necesitaron para llegar a cada usuario.
- iii. **Los datos de prueba:** Se decidió generar una red de 2 millones de usuarios con 16 amistades en promedio cada uno, lo que da 16 millones de amistades en total. La red se arma con un generador xorshift en lugar de rand, esto trae consigo 2 ventajas. La primera es que con la misma semilla sale exactamente la misma red y la segunda es que la red se puede construir de dos pasadas sin tener que guardar la lista completa de amistades en memoria mientras tanto. La red no se arma de cualquier forma, porque queríamos que se pareciera a una red

social de verdad, entonces cada nuevo usuario se conecta con alguno de los anteriores elegido al azar para así garantizar que todos queden conectados entre sí.

- iv. **¿Cómo se guardan los datos en memoria?:** Para la red que es grande todas las listas viven dentro de un solo bloque contiguo y cada usuario apunta a la parte que le toca dentro de ese bloque. Como las 32 millones de entradas de amistad son enteros de 4 bytes ese bloque pesa 128 megabytes. A eso se le suman 8 megabytes para llevar la cuenta de amigos por usuario y 16 megabytes para los punteros. La razón de hacerlo así es que BFS se pasa casi todo su tiempo recorriendo listas de amigos. Si los vecinos de un usuario están pegados en memoria, el procesador los trae al caché de una sola vez y el recorrido vuela. Si están regados, cada vecino cuesta una visita a memoria principal.

2. Estrategia de paralelización:

a. Histograma masivo:

- i. **¿Qué se paralelizó y qué no?:** El ordenamiento con Merge Sort se quedó secuencial. Lo que paralelizamos fue la construcción del histograma, que es la etapa donde el trabajo se puede repartir sin que un hilo dependa de otro.

- ii. **Directiva principal:** La región paralela del histograma es esta:

```
#pragma omp parallel for num_threads(numHilos) schedule(static)
\
    shared(histogramasLocales) firstprivate(minimo, maximo,
    ancho)
    for (h = 0; h < numHilos; h++) {
```

Un detalle que vale la pena señalar es que el ciclo no recorre los datos, recorre los hilos. A cada trabajador le corresponde

una vuelta del ciclo, y lo que recibe ese trabajador es un rango de cubetas del histograma. El primer hilo se encarga de las primeras cubetas, el segundo de las siguientes, etc hasta repartir las 100. La cláusula `firstprivate` da a cada hilo su propia copia del mínimo, el máximo y el ancho de cubeta. Son valores que solo se leen y nunca cambian, y darle una copia a cada uno evita que todos estén leyendo la misma posición de memoria, luego la cláusula `shared` sobre `histogramasLocales` deja que todos los hilos vean el mismo bloque de memoria, aunque cada uno escriba únicamente en su porción.

- iii. **¿Cómo evitamos las condiciones de carrera?:** Este es el núcleo de nuestra solución. Si todos los hilos escribiesen directamente al histograma global, 2 de ellos podrían intentar incrementar el mismo contador a la vez y se perdería una de las 2 cuentas. Lo resolvimos dándole a cada trabajador su propio histograma local.

```
int *histogramasLocales = calloc(numHilos * CUBETAS,
sizeof(int));
...
int *histogramaLocal = &histogramasLocales[h * CUBETAS];
```

Cada hilo habla de su histograma local y no toca el de otro. Si no hay dos hilos que escriben en la misma dirección de memoria, no hay condición de carrera que evitar. Al finalizar la zona paralela, el hilo principal recorre los histogramas locales y los acumula en el histograma global. Esa suma final es secuencial, pero como son sólo 100 cubetas por hilo, no pesa nada. Esta forma de resolverlo tiene la ventaja frente a usar una directiva `atomic` o una sección crítica en cada incremento. Con `atomic` los hilos se frenarían entre sí millones de veces, una por cada temperatura contada. Con los histogramas locales no se para nunca.

- iv. **¿Cómo encuentra cada hilo su parte de los datos?:** El hecho de que a un hilo se le asignen unas cubetas determinadas no le indica dónde están sus datos. Para eso volvemos a usar el arreglo que está ordenado. Cada hilo calcula la temperatura de inicio de su primer cubeta y la

temperatura de final de su última, y entonces usa una búsqueda binaria para encontrar esas dos posiciones en el arreglo.

```
int inicioRango = BuscarInicio(inicioTemperatura);
```

Con esto, cada hilo sabe exactamente en qué posición empezar y cuál parar y recorre únicamente su tramo del arreglo.

- v. **¿Para qué usamos `schedule static`?:** Cada hilo procesa el mismo número de cubetas, y el ciclo se ejecuta tantas veces como hilos haya. Los cuotas fijas, además de conseguirse, no cuestan nada. Un reparto dinámico sólo añadiría el gasto de andar pidiendo trabajo, sin ninguna ganancia. Es importante tener en cuenta que si bien la distribución de cubetas es equitativa, la cantidad de temperaturas que caen en cada cubeta no lo es. Las temperaturas se distribuyen uniformemente entre -100 y 100 por lo que en efecto los tramos resultan similares, pero no idénticos.

- vi. **La reducción del conteo final:** Al final el programa suma todas las cubetas para comprobar que no se perdió ningún dato.

```
#pragma omp parallel for reduction(+ : total)
num_threads(numHilos)
for (i = 0; i < CUBETAS; i++) {
    total += histograma[i];
}
```

La cláusula `reduction` le asigna a cada hilo su propia copia de la variable `total`, deja que cada quien vaya sumando por su lado y, al terminar, junta todas las copias en una sola. Sin esa cláusula habría condición de carrera, ya que todos estarían sumando a la misma variable al mismo tiempo.

b. Ruta mínima con BFS:

- i. **El problema de paralelizar BFS:** BFS secuencial usa una cola, y esa cola es justamente la que impide paralelizar. Si varios hilos sacan y meten usuarios de la misma cola, se pelearán por ella todo el tiempo, y el orden en que se atienden los usuarios no estará garantizado. La solución fue cambiar la forma en que se guardaba el trabajo pendiente. En vez de usar una cola, usamos dos arreglos, uno con los usuarios del nivel que se está procesando, y otro en el que vamos dejando los del siguiente nivel.

```
int *frontera = (int *)malloc((size_t)n * sizeof(int));
int *siguiente = (int *)malloc((size_t)n * sizeof(int));
```

Esto funciona debido a que todos los usuarios de un nivel están a la misma distancia del origen. Estas tareas son independientes entre sí y se pueden repartir entre los hilos sin problemas. Lo que debe respetarse es el orden entre niveles, eso es lo que hace la espera al cerrar cada nivel.

- ii. **La región paralela por nivel:**

```
#pragma omp parallel num_threads(numHilos)
{
    ...
} /* el cierre espera a todos y cierra el nivel */
```

Por cada nivel de recorrido se abre un equipo de hilos. Cuando la sección paralela se cierra, OpenMP automáticamente espera que todos los hilos terminen. Es esa espera la que mantiene la garantía de que BFS encuentra el camino mínimo, pues asegura que ningún hilo empiece el siguiente nivel antes de que el actual esté completo.

- iii. **¿Por qué usamos schedule dynamic?:**

```
#pragma omp for schedule(dynamic, 64) nowait
for (int i = 0; i < tamFrontera; i++) {
```

Aquí el tipo de reparto sí importa mucho, y es por el desbalance del que hablamos en la sección de datos. Un usuario cualquiera tiene en promedio 16 amigos, pero el más popular de nuestra red tiene 10131. Con un reparto fijo, el hilo asignado a ese usuario se quedaría trabajando solo, mientras que el resto ya han terminado y están esperando. Con `schedule` dinámico, los hilos van pidiendo trabajo conforme se desocupan, y el que agarró un usuario popular simplemente pide menos trabajo después. La carga se empareja por sí sola. El número 64 representa el tamaño del bloque que cada hilo solicita de una vez. Si pide uno por uno, costará más coordinar el trabajo que el trabajo mismo. Si se piden bloques muy grandes, volveríamos al problema de la distribución estática. El número 64 resulta de una media punto. La cláusula suprime la espera al final de ese ciclo. Cada hilo guarda sus resultados en su propio array, por lo que no necesita esperar a nadie en este punto. La verdadera espera es posterior, cuando se cierra la región paralela.

- iv. **¿ Cómo evitamos las condiciones de carrera?:** El riesgo aquí es la marca de visitado. Si dos hilos revisan al mismo usuario nuevo en el mismo instante, los dos podrían verlo como no visitado y los dos lo añadirían al siguiente nivel. Ese usuario saldría duplicado, y el recorrido se echaría a perder. Lo arreglamos con una operación indivisible.

```
int previo;
#pragma omp atomic capture
{ previo = visitado[v]; visitado[v] = 1; }
if (previo) continue;
```

La directiva `atomic capture` hace las dos cosas de una sola vez, lee el valor que tenía la marca y le pone uno. Es indivisible, por lo que ningún otro hilo puede quedar entre medio. El único que recibe un cero como valor previo es el hilo que da posesión del usuario, los demás reciben un uno y se van inmediatamente. Eso trajo una consecuencia que nos ahorró trabajo. Como cada usuario solo tiene un hilo, no hay que proteger las escrituras de invitadoPor y distancia para ese usuario, ya que nadie más las va a modificar.

- v. **La lectura previa que ahorra tiempo:**


```
int marca;
#pragma omp atomic read
marca = visitado[v];
if (marca) continue;
```

Antes de la operación de reclamo, se hace una simple consulta. Ya que las operaciones atómicas son bastante más costosas que una lectura normal, y en una red como la nuestra la mayoría de los vecinos de un usuario popular ya han sido descubiertos por alguien más. Con esta consulta previa se paga la operación completa únicamente cuando el usuario parece ser de verdad nuevo.

- vi. **El traslado al siguiente nivel:** Si cada hilo agregara sus hallazgos directamente al arreglo del nivel siguiente, tendrían que coordinarse en cada agregado y volveríamos a serializar el trabajo. En vez de eso cada hilo acumula sus hallazgos en un arreglo privado, y hasta el final reserva de un solo golpe todo el espacio que necesita.

```
int base;
#pragma omp atomic capture
{ base = tamSiguiente; tamSiguiente += nLocal; }
memcpy(siguiente + base, local, (size_t)nLocal * sizeof(int));
```

Con eso la coordinación entre hilos pasa de ocurrir una vez por usuario encontrado a ocurrir una sola vez por hilo y por nivel.

- vii. **La inicialización:**

```
#pragma omp parallel for schedule(static) num_threads(numHilos)
for (int i = 0; i < n; i++) {
    invitadoPor[i] = -1;
    distancia[i] = -1;
}
```

El ciclo llena dos arreglos de 2 millones de posiciones. Todas las vueltas tienen el mismo coste exacto, de forma que un reparto estático es el correcto y evita el gasto de coordinación que implicaría el dinámico.

3. Resultados y Métricas:

a. Resultados de Joel Jaquez:

i. Prueba de corridas:

```
joel@MacBook-Pro-de-Joel-2 examen-parcial-1 % python3 pruebas_rendimiento.py --integrante "Joel Jaquez"
```

```
#####
# CONSULTORIA HILO CONDUCTOR
# Examen Parcial 1 | Pruebas de speedup y eficiencia con OpenMP
# #####
```

```
INTEGRANTE : Joel Jaquez
FECHA      : 10/09/2026 14:22:48
EQUIPO     : macOS-26.6.2-arm64-arm-64bit-Mach-0
PROCESADOR : arm64, 14 nucleos logicos
COMPILADOR : Apple clang version 17.0.0 (clang-1700.6.3.2)
HILOS      : 1, 2, 4, 6, 8
```

PROBLEMA 1: Histograma masivo

```
Datos      : 1,000,000 temperaturas (semilla 20260909)
Se mide    : la fase paralelizada, el conteo por cubetas
Cada dato  : mejor tiempo de 5 repeticiones
```

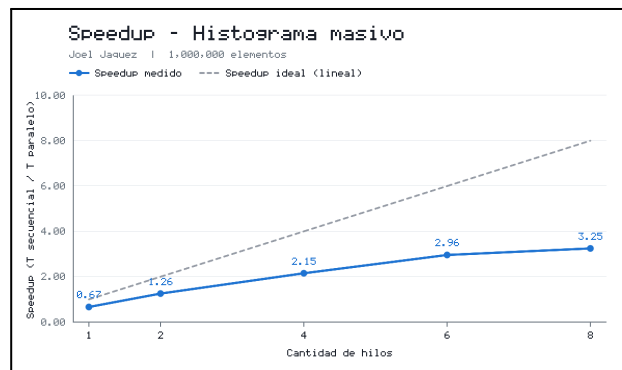
HILOS	TIEMPO (s)	SPEEDUP	EFICIENCIA	CORRECTO
1 (sec)	0.000496280	1.0000	100.0%	-
1	0.000745449	0.6657	66.6%	si
2	0.000394790	1.2571	62.9%	si
4	0.000230529	2.1528	53.8%	si
6	0.000167680	2.9597	49.3%	si
8	0.000152788	3.2482	40.6%	si

RESUMEN DE LA CORRIDA DE JOEL JAQUEZ

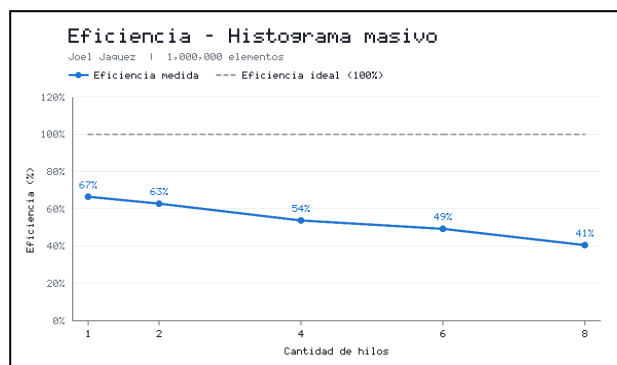
```
Histograma masivo
Mejor speedup      : 3.25x con 8 hilos
Eficiencia ahi     : 40.6%
Resultado igual al secuencial: si, en todos los casos
Ruta minima (BFS)
Mejor speedup      : 3.45x con 8 hilos
Eficiencia ahi     : 43.2%
Resultado igual al secuencial: si, en todos los casos
```

```
GRAFICAS GENERADAS en docs/resultados/joel-jaquez-20260910-142254
speedup_histograma-masivo.png
eficiencia_histograma-masivo.png
speedup_ruta-minima-bfs.png
eficiencia_ruta-minima-bfs.png
```

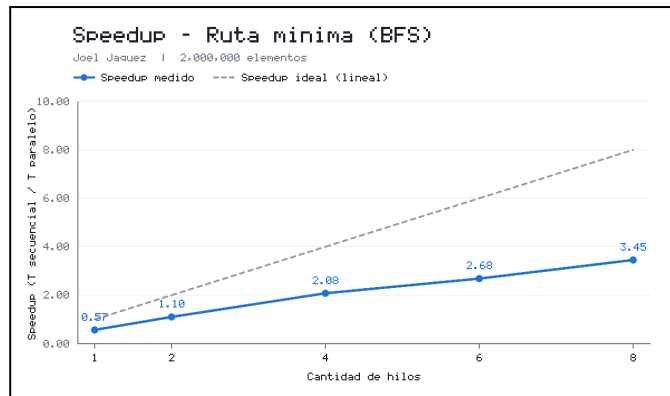
ii. Gráfica de speedup del histograma



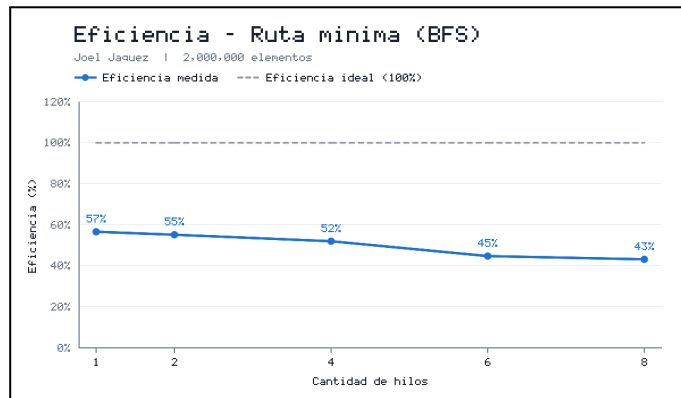
iii. Gráfica de eficiencia del histograma



iv. Gráfica de speedup de ruta mínima



v. Gráfica de eficiencia de ruta mínima



- vi. **Análisis de resultados:** Las 2 versiones paralelas empiezan perdiendo. Con un solo hilo el histograma llega a 0.67x y el BFS llega a 0.57x, es decir que corren más lento que la versión secuencial. Esto es lo que se esperaba porque con un hilo se paga todo el costo de armar el trabajo paralelo y no hay entre quienes repartirlo. En el histograma esos costos son los histogramas locales y las búsquedas binarias, ahora del lado del BFS son las operaciones atómicas y el manejo de las 2 fronteras. Con dos hilos las dos versiones ya ganan, 1,26x en el histograma y 1,10x en el BFS y desde ahí siguen subiendo hasta 3,25x y 3,45x con ocho hilos. En los dos problemas el punto donde empieza a rendir paralelizar queda entre uno y dos hilos.

Lo que más me llamó la atención fue el hecho de que las 2 curvas salieron tan parecidas. Son problemas distintos, uno está contando temperaturas sobre un array ordenado y el otro está iterando sobre un grafo de 2 millones de usuarios, pero ambos terminan cerca de 3.3x con la eficiencia en 41% y 43%. Eso indica que más que los algoritmos, es la máquina la que pone el límite. Los dos pasan la mayor parte del tiempo trayendo datos de memoria, en lugar de hacer cálculos, y el ancho de banda de la memoria se reparte entre todos los núcleos. Llegado un cierto punto ya no ayuda añadir más hilos, porque

todos acaban esperando por lo mismo. Por eso, también, la eficiencia desciende de forma pareja en los 2 casos y la distancia contra la línea ideal se va abriendo conforme se van agregando hilos. Con 8 hilos estoy usando apenas un poco más de 3 y los otros 5 se van en coordinación y esperas.

b. Resultado de Carlos Alburez:

i. Prueba de corridas:

```

Problems 5 Output Debug Console Terminal Ports
carlitos@MacBook-Air-de-Carlos examen-parcial-1 % python3 pruebas_rendimiento.py --integrante "Carlos Alburez"
PROBLEMA 1: Histograma masivo
=====
Datos      : 1,000,000 temperaturas (semilla 20260909)
Se mide    : la fase paralelizada, el conteo por cubetas
Cada dato  : mejor tiempo de 5 repeticiones

=====
HILOS      TIEMPO (s)      SPEEDUP      EFICIENCIA      CORRECTO
=====
1 (sec)    0.000450370    1.0000      100.0%         -
1          0.000680010    0.6623      66.2%          si
2          0.000350399    1.2853      64.3%          si
4          0.000203881    2.2090      55.2%          si
6          0.000221970    2.0290      33.8%          si
8          0.000189080    2.3819      29.8%          si
=====

PROBLEMA 2: Ruta minima (BFS)
=====
Datos      : 2,000,000 usuarios, 16 amistades promedio
Se mide    : el recorrido completo de la red
Cada dato  : mejor tiempo de 5 repeticiones

Construyendo la red social y midiendo...

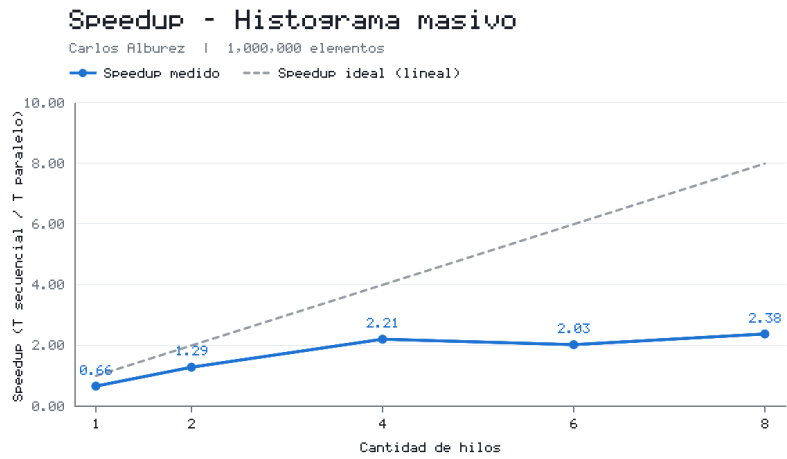
=====
HILOS      TIEMPO (s)      SPEEDUP      EFICIENCIA      CORRECTO
=====
1 (sec)    0.175237000    1.0000      100.0%         -
1          0.215472000    0.8133      81.3%          si
2          0.110807000    1.5815      79.1%          si
4          0.061358000    2.8560      71.4%          si
6          0.055687000    3.1468      52.4%          si
8          0.050346000    3.4807      43.5%          si
=====

Generando las graficas...

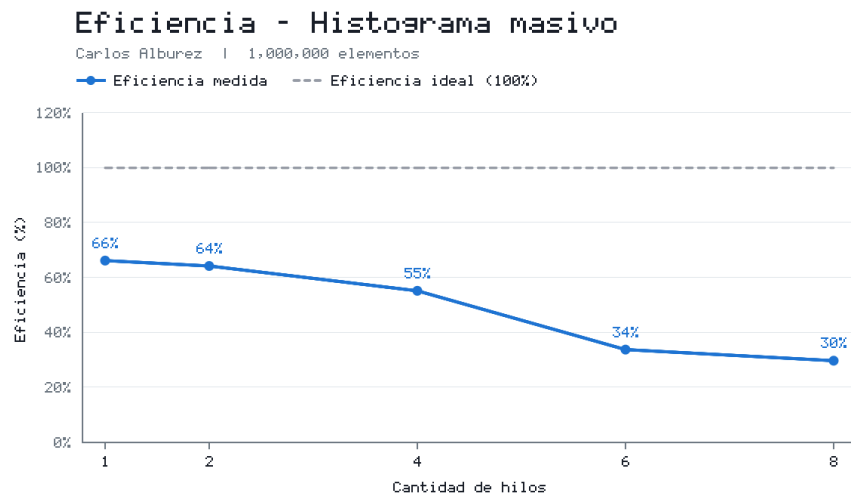
=====
RESUMEN DE LA CORRIDA DE CARLOS ALBUREZ
=====
Histograma masivo
  Mejor speedup      : 2.38x con 8 hilos
  Eficiencia ahi     : 29.8%
  Resultado igual al secuencial: si, en todos los casos
Ruta minima (BFS)
  Mejor speedup      : 3.48x con 8 hilos
  Eficiencia ahi     : 43.5%
  Resultado igual al secuencial: si, en todos los casos

```

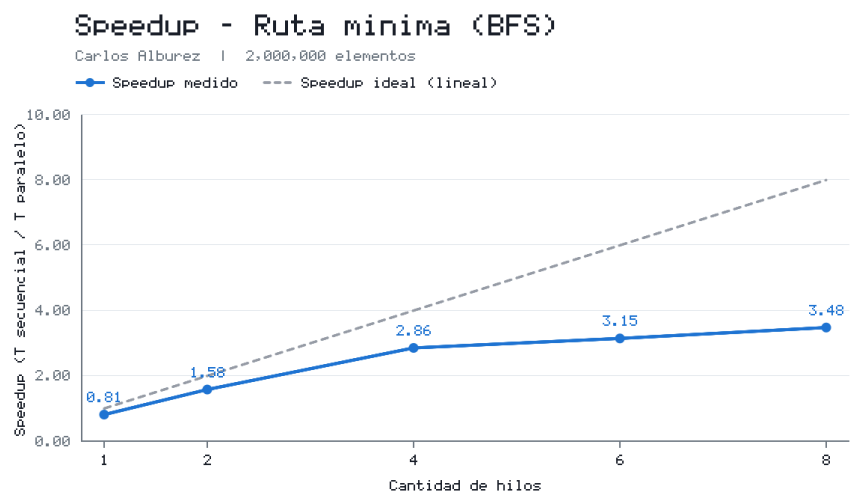
ii. Gráfica de speedup del histograma



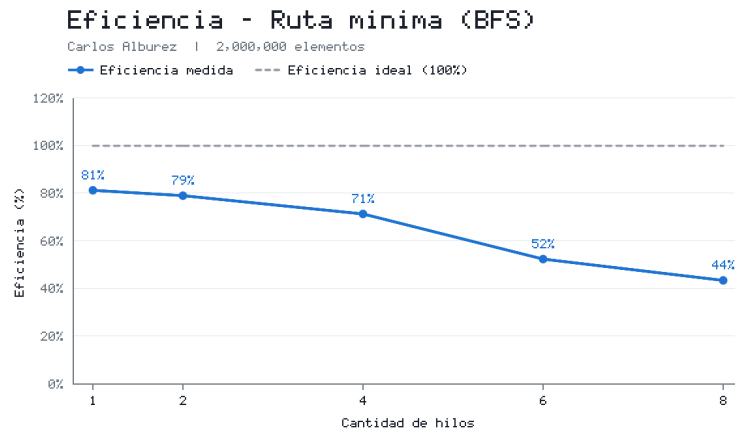
iii. Gráfica de eficiencia de ruta mínima



iv. Gráfica de speedup de ruta mínima



v. Gráfica de eficiencia de ruta mínima



vi. Análisis de resultados:

La paralelización del del histograma alcanzó un speedup máximo de 2.38x con 8 hilos, con una eficiencia de 30%. El speedup sub-lineal se debe a que la fase paralelizada es extremadamente rápida en secuencial (0.45ms), por lo que el overhead de crear y sincronizar hilos con OpenMP se vuelve comparable al cómputo útil. La eficiencia cae de 66% con 1 hilo a 30% con 8 hilos. El punto óptimo se ubica en 4 hilos (2.21x, 55% de eficiencia). En todos los casos el resultado paralelo coincidió exactamente con el secuencial.

Para la ruta mínima BFS, el problema escaló significativamente mejor, alcanzando un speedup de 3.48x con 8 hilos y una eficiencia del 44%. La razón principal es que el volumen de trabajo es alrededor de 390 veces mayor que el del histograma, lo que permite diluir el overhead de OpenMP y aprovechar mejor cada hilo. La eficiencia se mantuvo por encima de 70% en 4 hilos, que representa el punto con un speedup de 2.86x. A partir de ahí, la ganancia marginal disminuye, consistente con la Ley de Amdahl. Al igual que en el histograma, la correctitud fue verificada en todas las configuraciones de hilos.

En cuanto a las decisiones para mejorar el algoritmo secuencial, en el histograma, se utilizó `parallel for` con `schedule(static)` para distribuir equitativamente las 100 cubetas entre los hilos, junto con `shared` para el arreglo de histogramas locales y `firstprivate` para que cada hilo reciba su copia de los límites (mínimo, máximo, ancho). Cada hilo trabaja con su propio histograma local (`histogramasLocales[h * CUBETAS]`) y utiliza una búsqueda binaria para saltar directamente al segmento del arreglo ordenado que le corresponde, eliminando la necesidad de recorrer datos ajenos.

En el BFS, se empleó `schedule(dynamic, 64)` para balancear la carga irregular entre nodos con distinto número de vecinos, y cada hilo acumula sus descubrimientos en un buffer local privado que luego se vuelca a la frontera compartida con un único `atomic capture` por nivel, reduciendo la contención a una sola operación atómica por hilo en lugar de una por cada vecino visitado. Además, la lectura previa con `atomic read` antes del `atomic capture` sobre el arreglo de visitados funciona como un filtro rápido que evita operaciones costosas sobre nodos ya reclamados por otro hilo.

c. Resultado de Fernando Hernández:

i. Prueba de corridas:

```
examen-parcial-1 — -zsh — 106x60

#####
#
# CONSULTORIA HILO CONDUCTOR
# Examen Parcial 1 | Pruebas de speedup y eficiencia con OpenMP
#
#####

INTEGRANTE : Fernando Hernández
FECHA      : 11/09/2026 15:26:59
EQUIPO     : macOS-26.6.2-arm64-arm-64bit
PROCESADOR : arm64, 14 nucleos logicos
COMPILADOR : Apple clang version 21.0.0 (clang-2100.1.1.101)
HILOS      : 1, 2, 4, 6, 8

=====
PROBLEMA 1: Histograma masivo
=====
Datos      : 1,000,000 temperaturas (semilla 20260909)
Se mide    : la fase paralelizada, el conteo por cubetas
Cada dato  : mejor tiempo de 5 repeticiones

HILOS      TIEMPO (s)      SPEEDUP      EFICIENCIA      CORRECTO
-----
1 (sec)    0.000443730      1.0000      100.0%          -
1          0.000666280      0.6660      66.6%           si
2          0.000348380      1.2737      63.7%           si
4          0.000208230      2.1310      53.3%           si
6          0.000157480      2.8177      47.0%           si
8          0.000141861      3.1279      39.1%           si
-----

=====
PROBLEMA 2: Ruta minima (BFS)
=====
Datos      : 2,000,000 usuarios, 16 amistades promedio
Se mide    : el recorrido completo de la red
Cada dato  : mejor tiempo de 5 repeticiones

Construyendo la red social y midiendo...

HILOS      TIEMPO (s)      SPEEDUP      EFICIENCIA      CORRECTO
-----
1 (sec)    0.176866000      1.0000      100.0%          -
1          0.224710000      0.7871      78.7%           si
2          0.111901000      1.5806      79.0%           si
4          0.059621000      2.9665      74.2%           si
6          0.042058000      4.2053      70.1%           si
8          0.032317000      5.4728      68.4%           si
-----

Generando las graficas...

=====
RESUMEN DE LA CORRIDA DE FERNANDO HERNANDEZ
=====
Histograma masivo
  Mejor speedup   : 3.13x con 8 hilos
  Eficiencia ahi  : 39.1%
  Resultado igual al secuencial: si, en todos los casos
Ruta minima (BFS)
```



```
examen-parcial-1 — -zsh — 106x60

=====
PROBLEMA 1: Histograma masivo
=====
Datos      : 1,000,000 temperaturas (semilla 20260909)
Se mide    : la fase paralelizada, el conteo por cubetas
Cada dato  : mejor tiempo de 5 repeticiones

=====
HILOS      TIEMPO (s)      SPEEDUP      EFICIENCIA      CORRECTO
=====
1 (sec)    0.000443730    1.0000      100.0%         -
1          0.000666280    0.6660      66.6%          si
2          0.000348380    1.2737      63.7%          si
4          0.000208230    2.1310      53.3%          si
6          0.000157480    2.8177      47.0%          si
8          0.000141861    3.1279      39.1%          si
=====

=====
PROBLEMA 2: Ruta minima (BFS)
=====
Datos      : 2,000,000 usuarios, 16 amistades promedio
Se mide    : el recorrido completo de la red
Cada dato  : mejor tiempo de 5 repeticiones

Construyendo la red social y midiendo...

=====
HILOS      TIEMPO (s)      SPEEDUP      EFICIENCIA      CORRECTO
=====
1 (sec)    0.176866000    1.0000      100.0%         -
1          0.224710000    0.7871      78.7%          si
2          0.111901000    1.5806      79.0%          si
4          0.059621000    2.9665      74.2%          si
6          0.042058000    4.2053      70.1%          si
8          0.032317000    5.4728      68.4%          si
=====

Generando las graficas...

=====
RESUMEN DE LA CORRIDA DE FERNANDO HERNANDEZ
=====
Histograma masivo
  Mejor speedup   : 3.13x con 8 hilos
  Eficiencia ahi  : 39.1%
  Resultado igual al secuencial: si, en todos los casos
Ruta minima (BFS)
  Mejor speedup   : 5.47x con 8 hilos
  Eficiencia ahi  : 68.4%
  Resultado igual al secuencial: si, en todos los casos

GRAFICAS GENERADAS en docs/resultados/fernando-herandez-20260911-152704
speedup_histograma-masivo.png
eficiencia_histograma-masivo.png
speedup_ruta-minima-bfs.png
eficiencia_ruta-minima-bfs.png

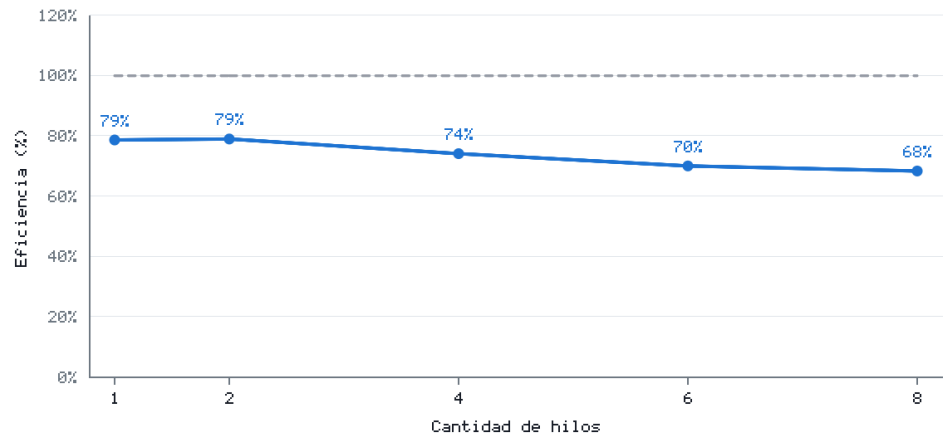
=====
ferahmz@Fernandos-MacBook-Pro examen-parcial-1 %
```

ii. Gráfica de eficiencia de ruta mínima

Eficiencia - Ruta minima (BFS)

Fernando Hernandez | 2,000,000 elementos

—●— Eficiencia medida --- Eficiencia ideal (100%)

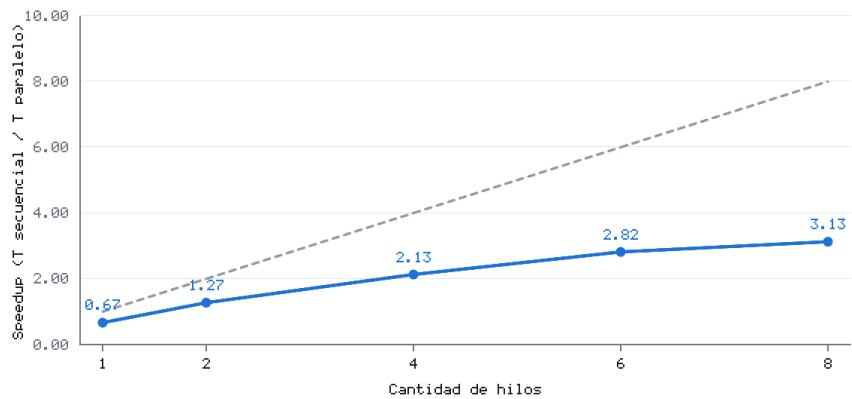


iii. Gráfica de speedup del histograma

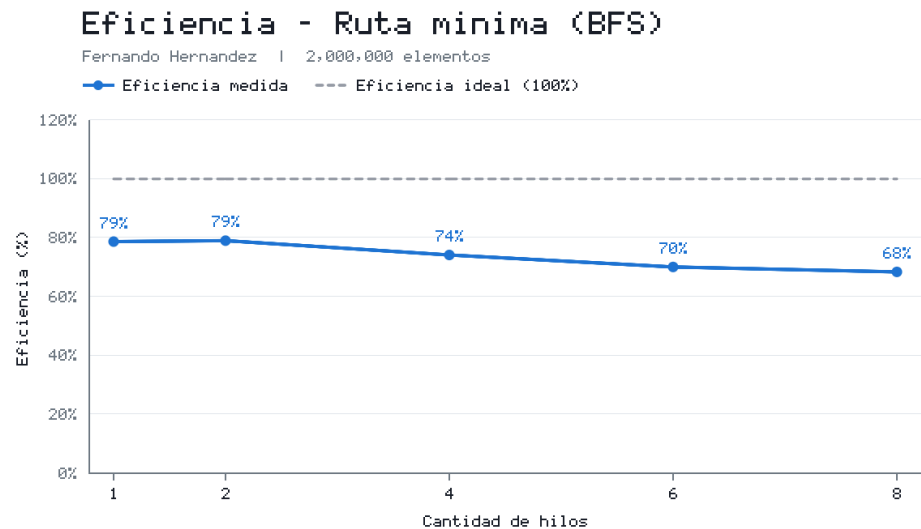
Speedup - Histograma masivo

Fernando Hernandez | 1,000,000 elementos

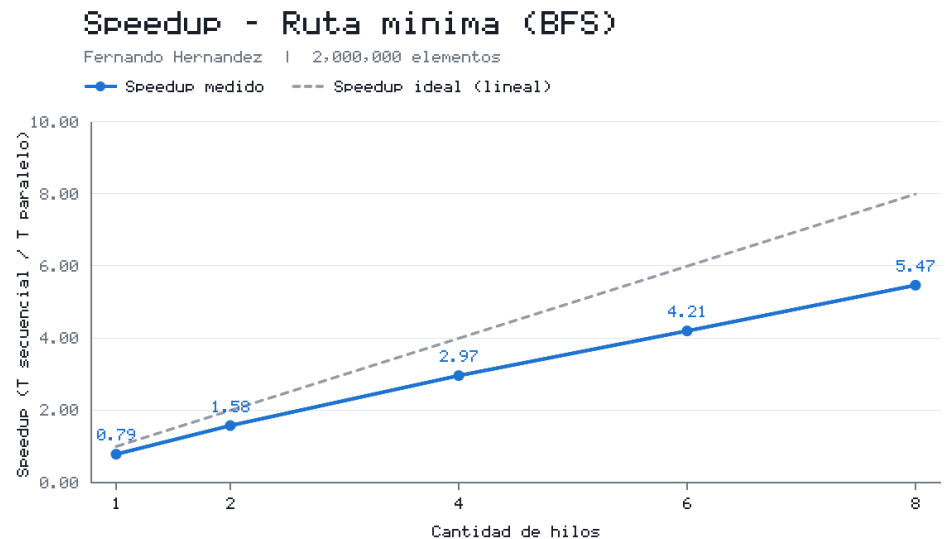
—●— Speedup medido --- Speedup ideal (lineal)



iv. Gráfica de eficiencia de ruta mínima



v. Gráfica de speedup de ruta mínima



vi. Análisis de resultados:

En mi máquina los dos problemas se separaron con claridad. El histograma llegó a 3.13x con 8 hilos y 39% de eficiencia, mientras que la ruta mínima con BFS alcanzó 5.47x con 68% de eficiencia. Es una diferencia mayor que la que reportaron mis compañeros, donde ambos problemas terminaron cerca de 3.3x, y creo que es el dato más informativo de mi corrida: muestra que el techo no es el mismo para los dos algoritmos, sino que depende de cuánto trabajo real tenga cada uno para repartir.

La forma de las dos curvas de eficiencia lo confirma. La del histograma baja de manera sostenida, de 67% a 39%, y la ganancia se va agotando: pasar de 6 a 8 hilos solo aporta 0.31x adicionales, apenas un 15% de lo que idealmente deberían aportar esos dos hilos extra. La del BFS, en cambio, se mantiene casi

plana entre 1 y 2 hilos (79% en ambos) y desciende muy despacio hasta 68%, con los últimos dos hilos aportando 1.26x, más del 60% de lo ideal. El histograma se queda sin trabajo que repartir mucho antes que el BFS, porque la fase que paralelizamos es el conteo por cubetas y esa fase dura menos de un milisegundo en secuencial, frente a los casi 0.18 segundos del recorrido completo de la red. Sobre un cómputo tan corto, el costo fijo de abrir la región paralela pesa tanto como el cómputo mismo, y ese costo no baja aunque se agreguen hilos.

El caso de un solo hilo mide ese costo fijo directamente, y también distingue a los dos problemas: el histograma pierde un 33% contra su versión secuencial y el BFS solo un 21%. Es decir que reservar los histogramas locales y hacer las búsquedas binarias sale proporcionalmente más caro que montar las dos fronteras y pagar las operaciones atómicas, precisamente porque hay mucho menos cómputo útil sobre el cual repartir ese gasto. En ambos casos la versión paralela es más lenta que la secuencial con un hilo, y el punto donde empieza a convenir queda entre uno y dos hilos.

La conclusión práctica para la consultora es que estos dos problemas no aceptan la misma recomendación. Para el BFS vale la pena usar todos los hilos disponibles, ya que a 8 hilos sigue rindiendo 68% y no da señales de haber tocado techo. Para el histograma, en cambio, la recomendación razonable es quedarse en 4 hilos, donde todavía rinde 53%; más allá de ahí se están gastando núcleos para ganar muy poco. La correctitud se verificó en todas las configuraciones: el resultado paralelo coincidió con el secuencial en los dos problemas y con todos los números de hilos probados.

3. Conclusiones

Lo primero que nos quedó claro es que paralelizar no sale gratis. En las tres máquinas, correr la versión paralela con un solo hilo salió más lento que el secuencial, entre 0.57x y 0.79x, porque ahí se paga todo el costo de armar el trabajo paralelo y no hay con quién repartirlo. Recién con dos hilos las dos versiones empiezan a ganar. Y lo que más pesó no fue qué tan bonito se deja dividir el algoritmo, sino cuánto trabajo real hay para repartir. El conteo del histograma se resuelve en menos de un milisegundo, así que abrir la región paralela cuesta casi lo mismo que el trabajo en sí, y por eso a 8 hilos su eficiencia quedó entre 30% y 41% en los tres. El BFS, que tiene como 400 veces más trabajo, se mantuvo entre 43% y 68%. La regla que nos llevamos es simple: antes de paralelizar hay que medir cuánto dura la parte que uno quiere repartir, porque si dura menos de un milisegundo el overhead se la come.

Lo segundo es que los dos programas dieron siempre el mismo resultado que el secuencial, en todas las pruebas y con todos los números de hilos, y lo logramos sin frenar a los hilos entre sí. Cada problema pidió una solución distinta pero con la misma idea detrás: la mejor sincronización es la que no hace falta. En el histograma ni siquiera hubo que sincronizar, porque al darle a cada hilo cubetas distintas y su propio tramo del arreglo, nunca hay dos

escribiendo en el mismo contador. En el BFS sí era inevitable que dos hilos se pelearan por el mismo usuario, entonces usamos una operación indivisible, pero le pusimos una lectura previa para no pagarla de gusto y movimos la coordinación del encolado a una sola vez por hilo y por nivel. Si lo hubiéramos resuelto poniendo un atomic en cada incremento, habríamos serializado justo la parte que más se repite y la versión paralela habría terminado más lenta que la original.

Lo tercero es que el mismo código, con los mismos datos y la misma semilla, nos dio resultados bastante distintos: el BFS a 8 hilos dio 3.45x, 3.48x y 5.47x según la máquina, y el histograma entre 2.38x y 3.25x. Esa diferencia es demasiado grande para ser ruido, y viene del hardware, de cuántos núcleos hay, del caché y sobre todo del ancho de banda de memoria, que es lo que los dos algoritmos terminan saturando porque se la pasan trayendo datos más que calculando. Aun así, el orden entre los dos problemas se repitió en las tres máquinas: el BFS siempre escaló igual o mejor que el histograma. Para una consultora eso significa que un número de speedup no dice nada si no se dice en qué máquina se midió, y que las pruebas hay que hacerlas donde el programa va a correr de verdad, no en la laptop del que lo escribió. Lo que sí se lleva de una máquina a otra no son los números, sino el criterio: cuánto trabajo hay para repartir y cómo evitar sincronización que no hace falta.

hacerse sobre el hardware donde el software va a correr de verdad, no sobre el del desarrollador. Lo que sí es transferible entre máquinas no son los números, sino el criterio de diseño cuánto trabajo hay que repartir y cómo se evita la sincronización innecesaria, y eso es precisamente lo que un ingeniero en HPC aporta por encima de escribir un ciclo for.